Plan and Timeline

Target: apply to the Anthropic Fellows Program, **ML Systems & Reinforcement Learning** posting (RL workstream: "creating RL environments ... within your domain of expertise"), for the cohort expected to start in January 2027. Applications there are reviewed on a rolling basis, so 2026-11-30 is the latest date, not the goal: apply as soon as the results table is in the README. Phases 4 and 5 continue after the application is submitted. Progress is logged daily in JOURNAL.md.

Scope for the application (decided 2026-11-10)

On 2026-11-10 no simulator code existed yet, five days into Phase 1. The scope was cut so that one call flow with real results is the bar for applying. A results table in the README outranks everything else.

Must have

- `ivrsim` loader, load report and session engine
- One pack: Not Dead Yet Pension Line (bilingual, decoys, planted smells)
- Bug injector (14 operators), every mutant hand-reviewed
- Grader for scripted calls
- Path-walker baseline, scored
- Claude API walker, black box, scored
- README rewritten around the results table, with a one-command reproduction
- Journal caught up and kept daily

Should have (in this order, as time allows)

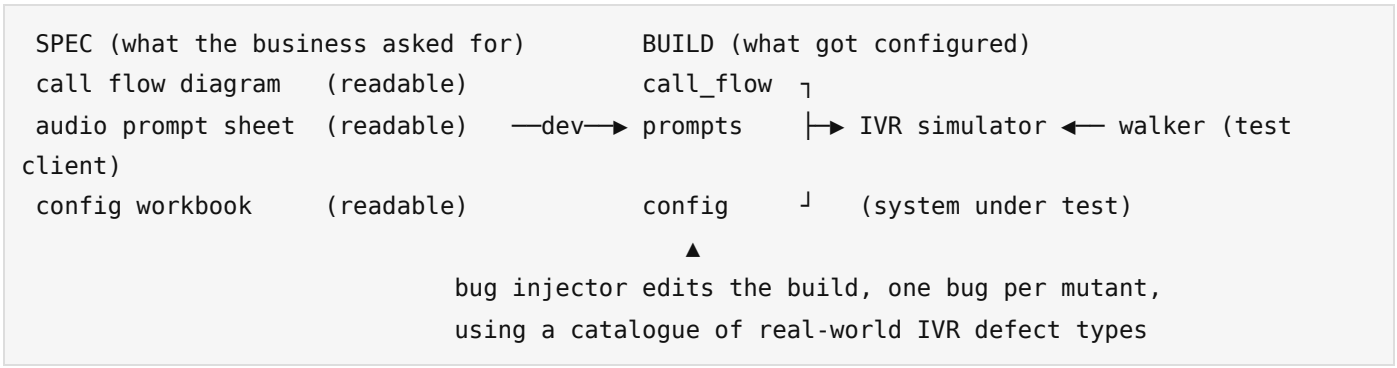
- White-box mode
- The correct build served, for a false-positive rate
- Decoy-error rate
- MCP harness, with Claude Code as a walker
- Reward-hacking log
- Developer consoles (manual caller, record a call as a test)

Deferred until after the application

- Call flows 2 and 3
- OS matrix in CI
- Docker and GHCR (tests are data, so the grader never executes walker code)
- GitHub Pages dashboard and the nightly run
- The small open model, unless it runs locally with little effort

Design

Real IVRs are a fixed engine running configuration, and most defects are translation errors between the requirement documents and that configuration. bug-bash reproduces that structure:



- **Simulator** (system under test): loads a build and plays the IVR side of a call. It holds the call state, plays prompts and waits for caller input. It is `ivrsim/`, a self-contained package that never imports `bugbash` (design: IVRSIM_DESIGN.md).
- **Load report and baseline**: every load reports errors (Architect would refuse to publish), warnings, and every difference from a named baseline. A clean load (0 errors, warnings acknowledged, 0 differences) certifies the correct build, and the bug injector runs only on a certified build.
- **Walker** (test client): the caller. Decides what to press and what to expect. Walkers compete in the bug bash: the rule-based path-walking test generator (non-AI baseline) and language models.
- **Test = scripted call**: time of call, caller inputs, expected prompts and outcome. Tests are data, not code, so the grader never executes walker-written code.
- **Scoring rule**: a test earns credit only if its expected result matches the reference build and differs on the buggy build.
- **What the model receives**: the readable spec documents plus a phone line into one build (buggy, or occasionally the correct one, without being told which). Two modes: black box (phone line only) and white box (phone line plus the build's configuration files). The baseline diff and the hidden answer keys (decoys, design smells) are never in what the model receives.
- **Joke decoys**: where a famous joke suggests a value, the spec states a different one. A model that guesses from the joke instead of reading the spec is wrong, measurably.
- **Design smells**: confusing but spec-valid design (a "main menu" key that behaves differently in different places). Not bugs: reporting one as a bug is a false positive.
- **Known limitation**: this tests IVR logic, not a real telephony platform. In exchange every call is fast and deterministic, which makes large-scale scoring and training possible.

All call flows, prompts and data are fictional and written for this project. No client material is used.

Simulator interface (proposal, owned by David)

```
        pack, report = load_pack(build_dir)          # the loader takes file contents; the
report is always written
        session = IvrSession(pack, when=datetime(...), world={...})
        session.next()        # -> Prompt(id, lang, text) | WaitInput(kind, min, max,
terminator)
                              #    | Transfer(queue) | AgentSays(text) | Disconnect(reason)
        session.send(event)   # event: Digits("2") | Digits("000101#") | NoInput() | Hangup()
| Say(text)
        session.trace         # every event, including nodes the caller never hears

        run_call(pack, when, world, inputs) -> list[Event]   # scripted convenience wrapper
```

The step-by-step session lets a model react to what it hears, like a real caller. `run_call` serves the grader and the path-walker. `next()` returns only what the caller perceives (black box); `trace` holds everything (white box and debugging). Tests assert prompts by ID, and the grader compares the text heard with the reference build's text for that ID, so a swapped or corrupted prompt text is caught. No wall clock, randomness, threads or sockets inside the engine: the same code can later run in a browser.

Division of work

David	Claude (pair)
Bug catalogue (IVR defects seen on every project)	Bug injector in `ivrsim`, from the catalogue: certified baseline only, one change per mutant, diff stored
Data file formats (call flow, prompts, config)	Spec renderer: build files -> readable spec documents
IVR simulator: loader and load report, session engine, automated service officer	Grader for scripted calls, mutant caching, CI
Not Dead Yet Pension Line call flow	Model harness (Phase 2)
Path-walker baseline (own prior work, rewritten for the new build format)	Engine self-tests alongside the simulator
Joke decoys and planted design smells, with hidden answer keys	Developer consoles: manual caller and manual service officer; record a call as a test
	Guard tests (generic engine, fictitious data) and a pre-commit hook

Timeline

Phase	Dates (2026)	Deliverable	Done when
0. Scaffolding	Nov 3	Repo, CI, first code-level task	**Done**

Phase	Dates (2026)	Deliverable	Done when
1. IVR environment	Nov 10 - Nov 18	Simulator, one call flow, bug injector, grader, path-walker baseline	Every mutant reviewed against the spec text; path-walker scored per operator; CI green with guards required
2. The bug bash	Nov 19 - Nov 26	Claude API walker, black box; then the should-have list in order	Repeatable scored runs from one command, with attempt logs
3. Results	Nov 27 - Nov 29	Results table, failure analysis, README rewrite	A reader can reproduce the table from the README
Apply	**Nov 30 at the latest**	Application submitted with repo link	
4. Training (stretch)	Dec 1 - Dec 14	LoRA rejection-sampling fine-tune of open model	Before/after on held-out call flows, reported honestly
5. Write-up	Dec 15 - Dec 21	REPORT.md	Motivation, method, results, limitations, next steps
6. Python upgrade	Dec 22 - Dec 28	Before/after comparison, 3.13 vs the current 3.15 release: the same recorded test scripts re-graded on both, the self-tests, and timings; then the lock moved to 3.15 with the smallest dependency changes needed	Scores identical on both; runtime and configuration differences measured and explained; REPORT.md gains an upgrade section

These dates replace the build plan in section 7 of IVRSIM_DESIGN.md.

Phase 1 steps

1. **Nov 10 - 11**: bug catalogue, data file formats, loader and load report (David); guard tests and pre-commit hook (pair)
2. **Nov 12 - 13**: session engine (David); engine self-tests, spec renderer (pair)
3. **Nov 14**: Not Dead Yet Pension Line, bilingual EN/FR, full build + spec, with joke decoys and planted design smells
4. **Nov 15 - 16**: bug injector in `ivrsim` (14 operators, certified baseline only); hand review of every mutant
5. **Nov 17**: grader for scripted calls, automated service officer
6. **Nov 18**: path-walker baseline plugged in and graded; guards required in CI

Phase 2 and 3 steps

1. **Nov 19 - 21**: Claude API walker, black box, with attempt logs
2. **Nov 22 - 24**: white-box mode, the correct build served, decoy-error rate
3. **Nov 25 - 26**: MCP harness and Claude Code as a walker; reward-hacking log
4. **Nov 27 - 28**: results table, failure analysis, README rewrite
5. **Nov 29**: buffer

If a step slips, the should-have items give way first, from the bottom of the list up. The must-have list does not shrink.

Phase 6 steps

1. **Before (3.13.9):** record the environment (Python, pytest and every installed package version). Re-grade every recorded test script from Phase 3: the path-walker's and the models'. Run the self-tests, and time each step. The models are not called again; their test scripts are data, so this isolates Python from model randomness.
2. **Upgrade:** move CI, `requires-python` and the dev machines to 3.15. Change a dependency only if it fails on 3.15, and record why.
3. **After (3.15):** repeat step 1 exactly.
4. **Compare:** every score must be identical, mutant by mutant. Report the runtime difference, and any warning or configuration change the upgrade needed. The comparison goes into REPORT.md as an upgrade section.

Engineering practice

The environment is also a showcase of how it is built: tested, gated and reproducible. Items marked *deferred* wait until after the application (see Scope for the application).

1. **CI/CD.** Extend `ci.yml`:
    - guards 1-4 run as **required checks**, matching the pre-commit hook;
    - *deferred:* an **OS matrix** (ubuntu, windows, macos) next to the Python matrix, since the engine is meant to be portable;
    - validated mutants **cached** with `actions/cache`, keyed on a hash of the pack;
    - a **score regression gate**: every pull request re-grades the path-walker on every call flow, fails if recall drops, and posts the score change as a PR comment;
    - *deferred:* a **nightly scheduled run** of the full baseline bug bash, results uploaded as a workflow artifact.
2. **Docker (*deferred*).** A `Dockerfile` for the grader sandbox, built in CI and published to GHCR; the CI grading job runs inside that image. Needs Docker installed on a dev host first (see the Day 1 journal blocker).
3. **Quality metrics (history in Phase 2; dashboard *deferred*).** Every scored run appends to a metrics history: recall, precision, decoy-error rate, hidden-test gaps found, suite runtime, and flake rate from repeat runs (expected 0, since the engine is deterministic; measuring it is the point). CI publishes the trends as a small GitHub Pages dashboard.
4. **Agent harness over MCP (Phase 2, should-have).** Expose `IvrSession` as an MCP server with tools to dial, press digits, listen and hang up. Black-box mode is then tool access only, and Claude Code runs as a walker next to the Claude API model and the small open model. The same harness can later run GitHub Copilot as a walker (links to IVR_AgentBench).
5. **Review log of AI-written work (all phases).** The journal template gets a **Caught in review** line: AI-written code or tests that were rejected or corrected, and why (for example a test that passed but verified nothing). It records the human-review half of the division of work above.

Later (out of scope for now)

- **Compatibility testing**: until the system works, everything is locked to one version: Python 3.13.9, pytest 8.4.2, Ubuntu 24.04 LTS in CI (decided 2026-11-10). A version changes only to get past a reproduced problem, in the smallest change that does it. Phase 6 then moves the lock to Python 3.15 (released 2026-10-09), proving with a before/after run that the upgrade changes no score. Other Ubuntu versions, Windows and macOS, and Pyodide come after that.
- **Spec-vs-spec task type**: inject inconsistencies between the documents themselves (call flow vs prompt sheet vs workbook) and score how many a model finds before anything is built.
- More domain clusters: network security config, protocol decoding, embedded state machines, software lifecycle.
- **Design-evaluation bonus question**: "evaluate the IVR you tested, against the spec and against industry norms", answered as structured findings (node, smell class, reason) and scored against the hidden smell list, with no judge.
- **Fix task**: the model proposes a change to the build; it scores if the fixed build passes the hidden tests. A known mutant's fix is its inverse diff.

Open questions

- Score per mutant vs per call flow: report both.
- Should a walker also name the defect (node + description), scored separately from the test call?
- How often should the correct build be served, so "no bug found" is a rewarded answer and flooding with reports is not? (The clean-load certificate guarantees the build served is in fact correct.)
- Report a decoy-error rate (expectations that matched the joke, not the spec) alongside recall and precision?